IMEN891K / IDSC501 / AXGS-501 · DATA ENGINEERING · FALL 2026

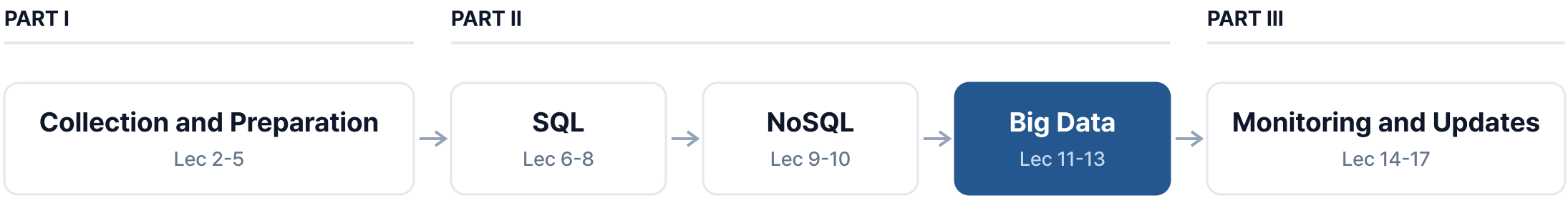
12.

Big Data II: Data Warehousing

Sangwoo Mo · POSTECH

Industrial & Management Engineering

Where We Are


PART I
PART II
PART III
Collection and Preparation
Lec 2-5
SQL
Lec 6-8
NoSQL
Lec 9-10
Big Data
Lec 11-13
Monitoring and Updates
Lec 14-17

Recap: Distributed Batch Processing

A distributed program

```
(sales.groupBy("product_id")
      .sum("amount")
      .show())
```

A distributed program still needs infrastructure, storage and a way to run again

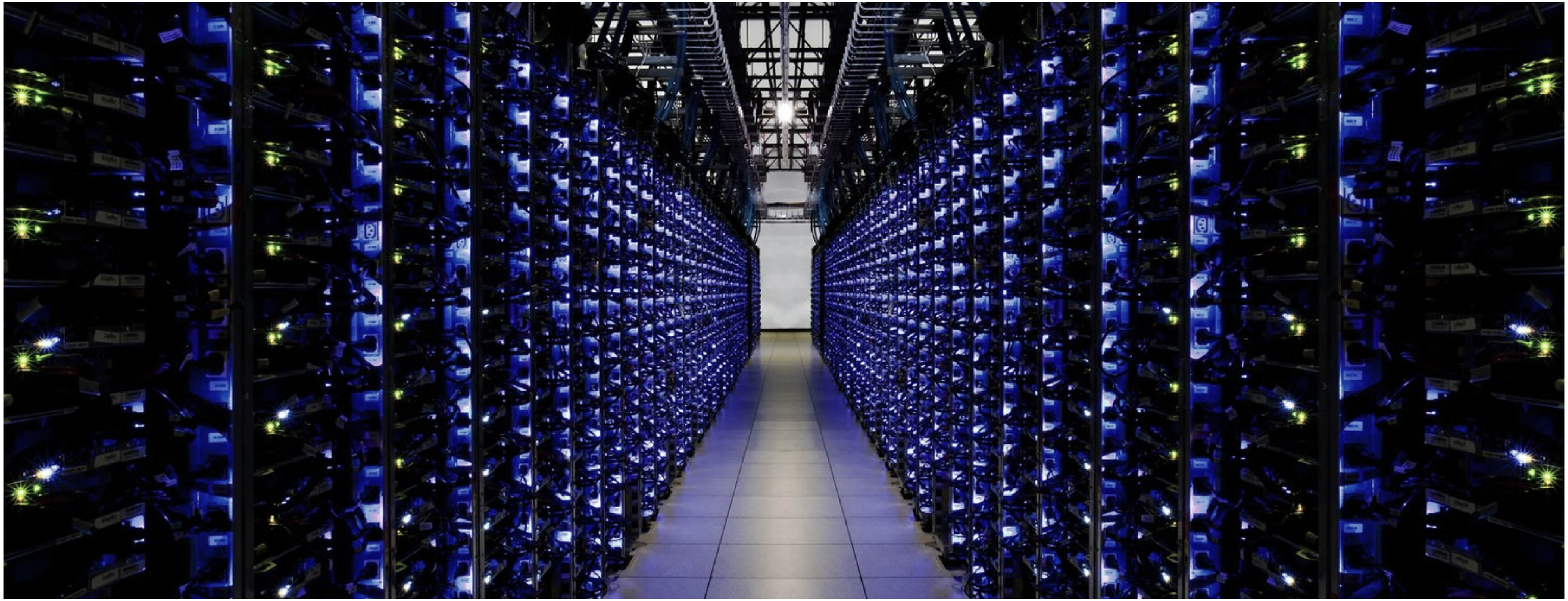
Outline: Big Data II

01	**Cloud and Serverless**	Who operates the infrastructure?
02	**Analytical Warehouses**	How are many rows read and summarized?
03	**Lakes and Lakehouses**	Where do files become tables?
04	**Data Orchestration**	Which task runs after which, and safely again?

01 · Cloud and Serverless

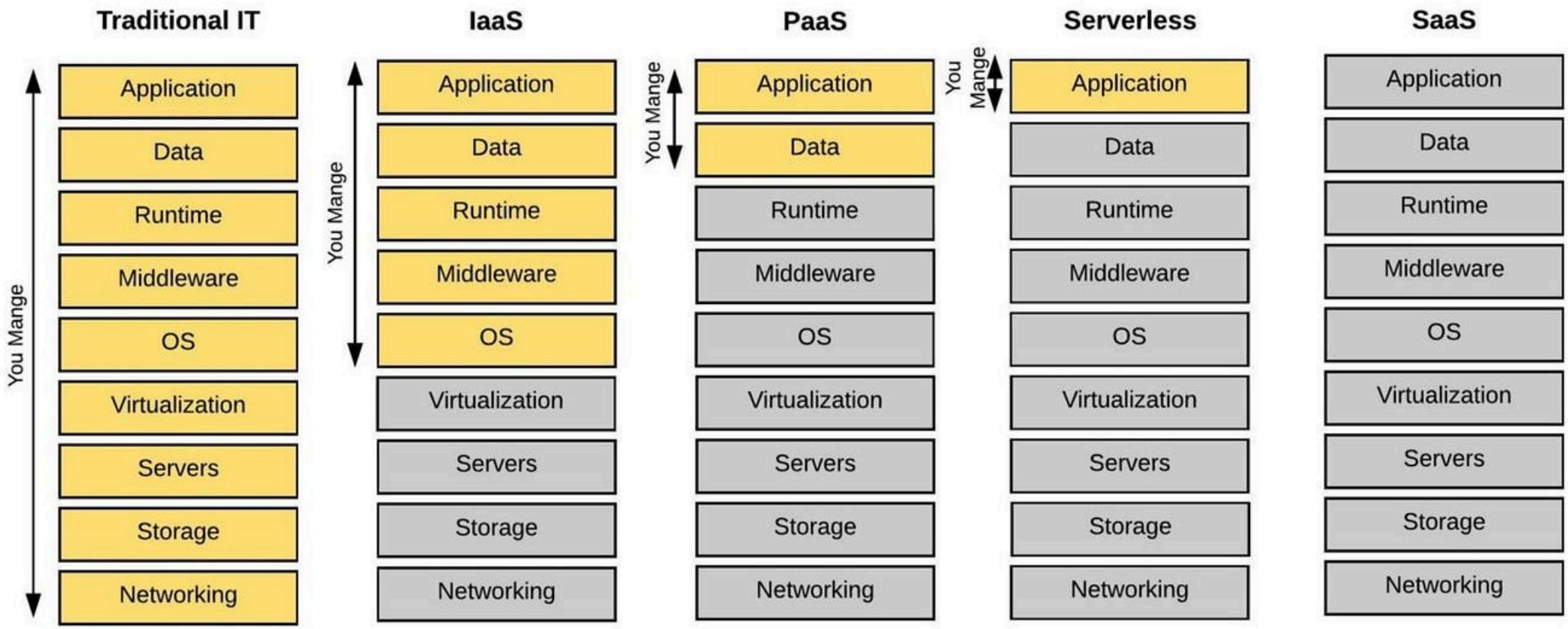
01	**Cloud and Serverless**	Who operates the infrastructure?
02	**Analytical Warehouses**	How are many rows read and summarized?
03	**Lakes and Lakehouses**	Where do files become tables?
04	**Data Orchestration**	Which task runs after which, and safely again?

Why Managed Infrastructure



The cloud still runs on physical servers that somebody must operate

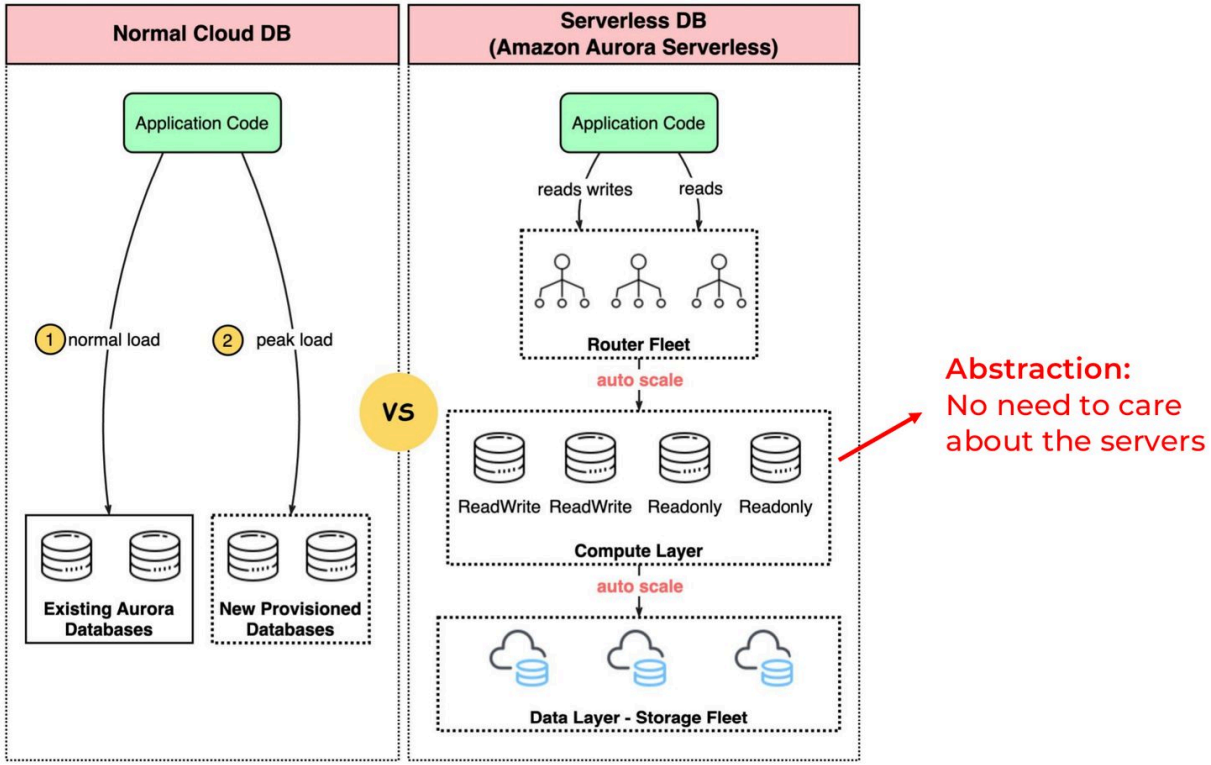
Cloud Service Models



Moving right transfers more infrastructure operations to the provider

Serverless Computing

Serverless transfers server provisioning and operation to a managed service



The service scales servers behind the application interface

Milestones in Managed Analytics

2004
MapReduce paper
batch on clusters

2009
Spark at Berkeley
reusable chains

2010
BigQuery preview
SQL as a service

Today
managed compute
and shared storage

Analytics shifted from operating clusters to submitting SQL to a service

BigQuery as a Managed Service



Ingest

- Batch & Streaming
- Structured & Semi-structured



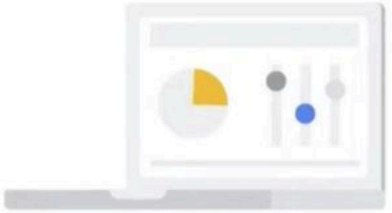
Store

- Up to Petabytes of data
- Datasets & tables
- Federate data



Analyze

- Super-fast SQL queries
- Aggregate
- Transform with DML



Visualize

- Export and connect to other tools more easily

Analysts submit SQL while the service manages distributed execution

Spark and BigQuery

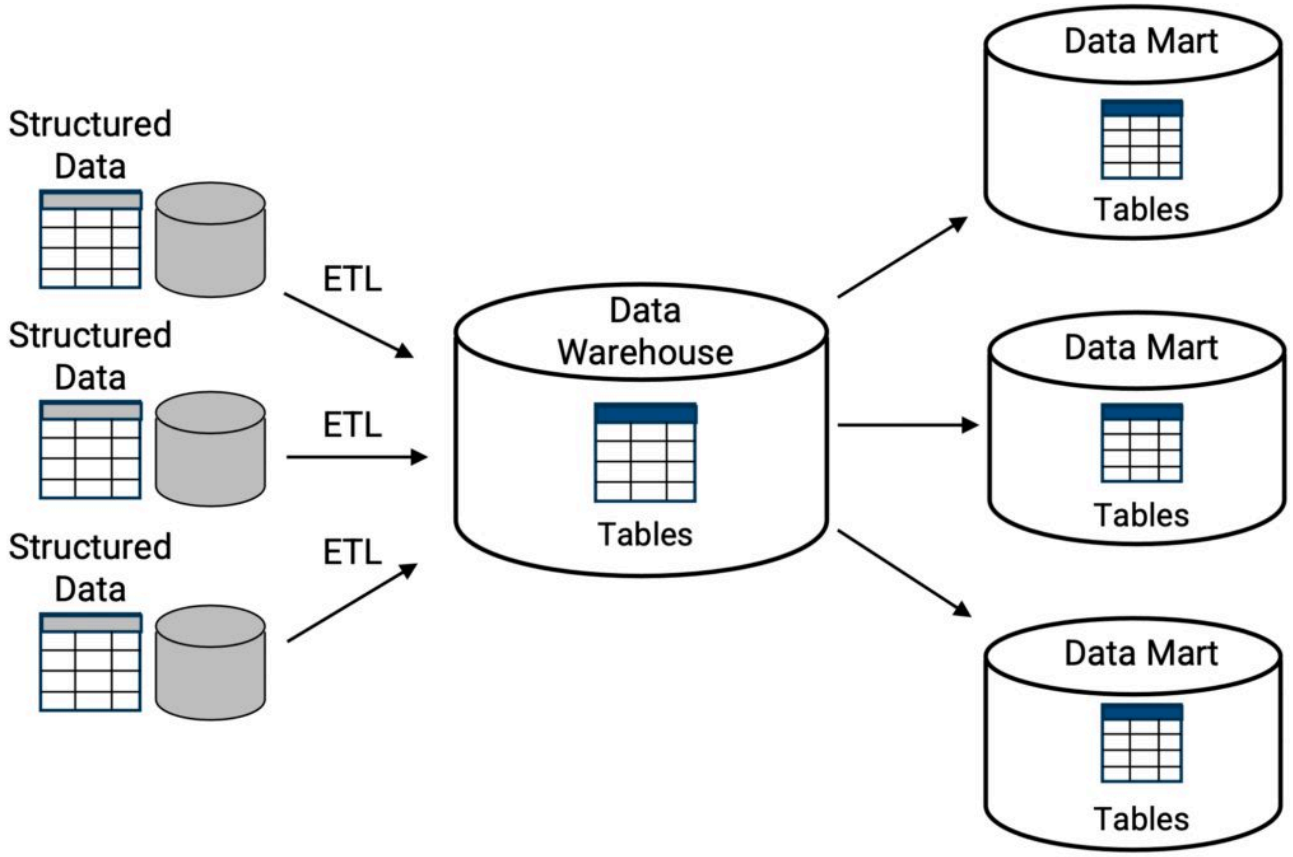
Tool	User expresses	Typical responsibility
Spark	A distributed program	Choose transformations and execution environment
Managed Spark service	A Spark program	Use a provider-managed execution environment
BigQuery	An analytical SQL query	Define tables and the analytical question

An execution engine and a managed analytical service solve different parts of the workload

02 · Analytical Warehouses

01	Cloud and Serverless	Who operates the infrastructure?
02	**Analytical Warehouses**	How are many rows read and summarized?
03	Lakes and Lakehouses	Where do files become tables?
04	Data Orchestration	Which task runs after which, and safely again?

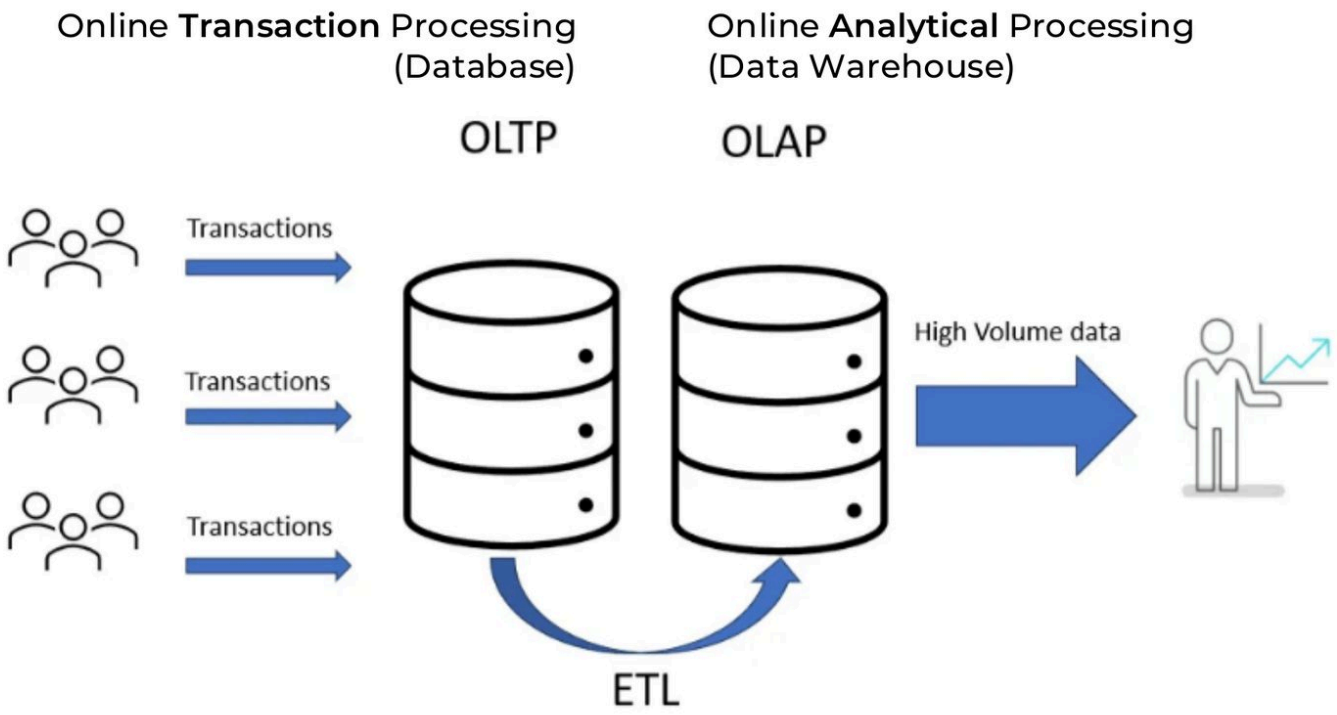
Why an Analytical Warehouse



Shared analytical tables connect multiple sources to consistent departmental reports

OLTP and OLAP

Online Transaction Processing (OLTP) and Online Analytical Processing (OLAP)



OLTP typically uses row stores; OLAP typically uses column stores

Workloads Shape Physical Storage

Workload	Read / write pattern	Typical physical choice
OLTP	Find and update a few complete records	Row store + key indexes
OLAP	Scan a few columns across many records	Column store + compressed scans
Mixed workload	Fresh writes and large analytical reads	Separate replicas or a hybrid design

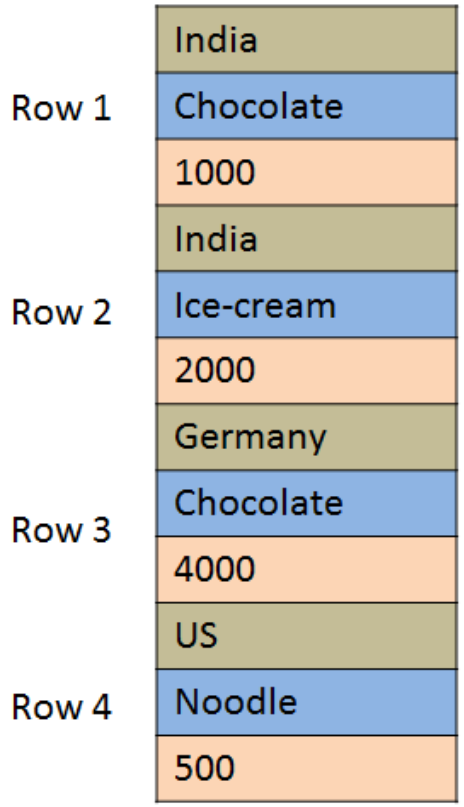
OLTP favors record access; OLAP favors scanning selected attributes

The Same Values on Disk

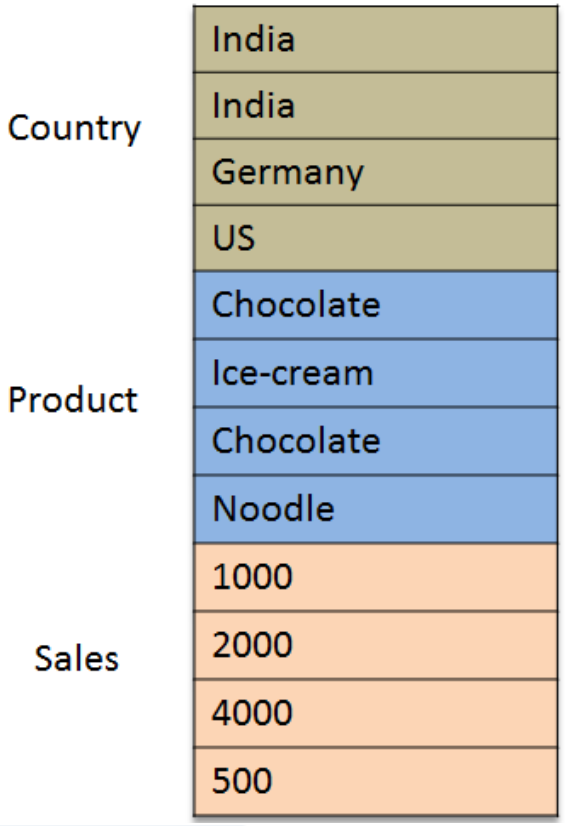
Logical table

Country	Product	Sales
India	Chocolate	1000
India	Ice-cream	2000
Germany	Chocolate	4000
US	Noodle	500

Row store (OLTP)



Column store (OLAP)

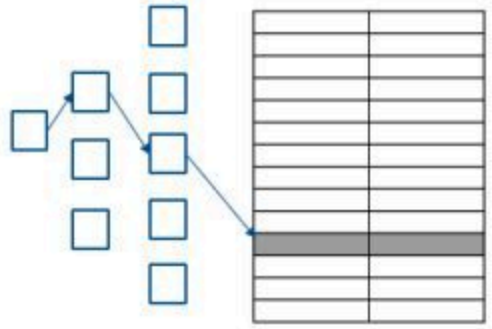


Twelve values retain their meaning while their physical order changes

Columnar Storage

The access pattern explains the usual OLTP / OLAP storage choice

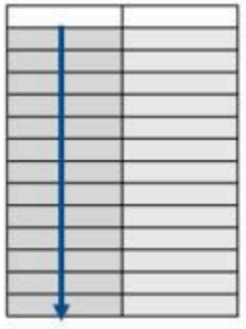
Row-oriented storage & execution



```
SELECT * FROM orders
WHERE orderid = $1
```

At scale: Fast on OLTP, Slow on OLAP

Column-oriented storage & execution



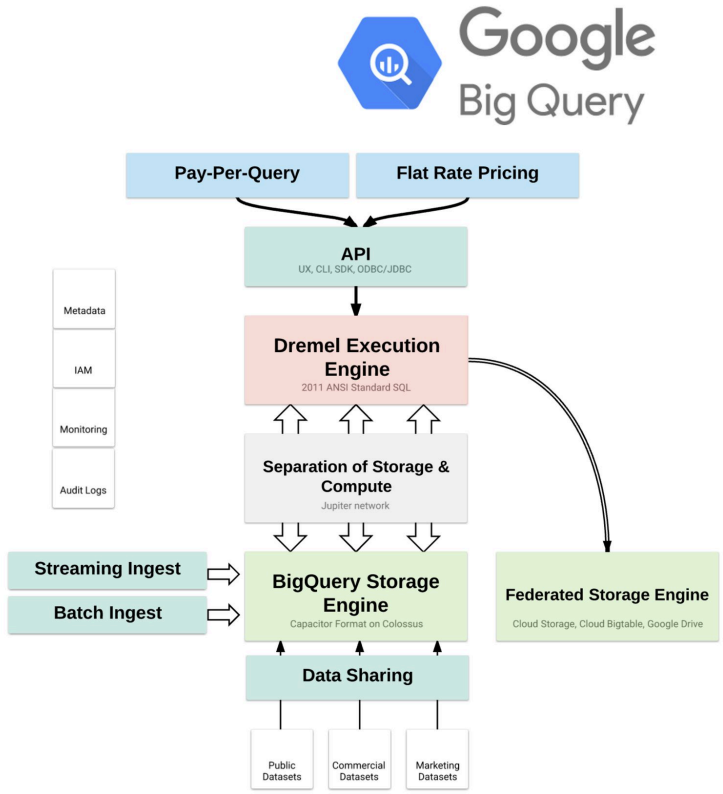
```
SELECT productid, count(*) FROM orders
GROUP BY 1 ORDER BY 2 DESC LIMIT 10
```

At scale: Slow on OLTP, Fast on OLAP

Row stores gather records; column stores scan and compress selected attributes

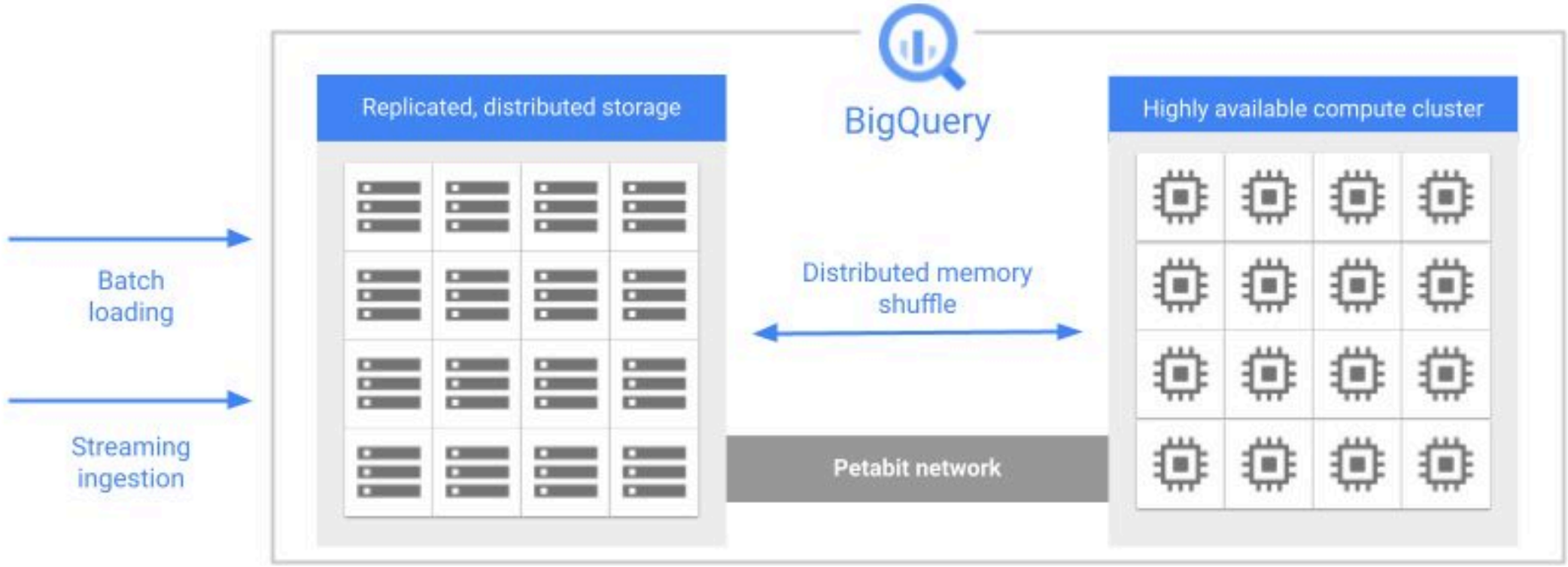
Inside BigQuery

Historical architecture: follow the query and its data



SQL execution, columnar storage and the network form separate service layers

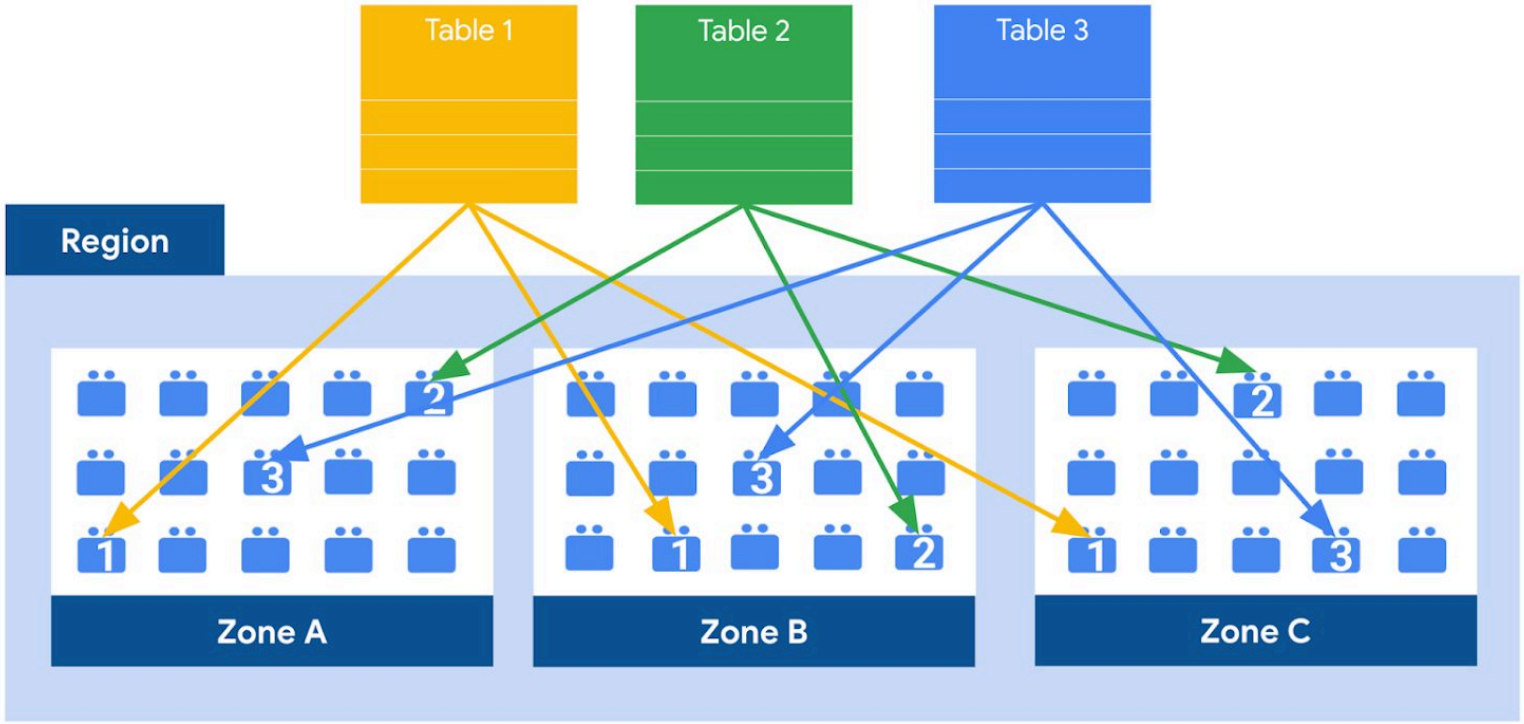
BigQuery Storage and Compute



Persistent table storage and query execution are managed as separate layers

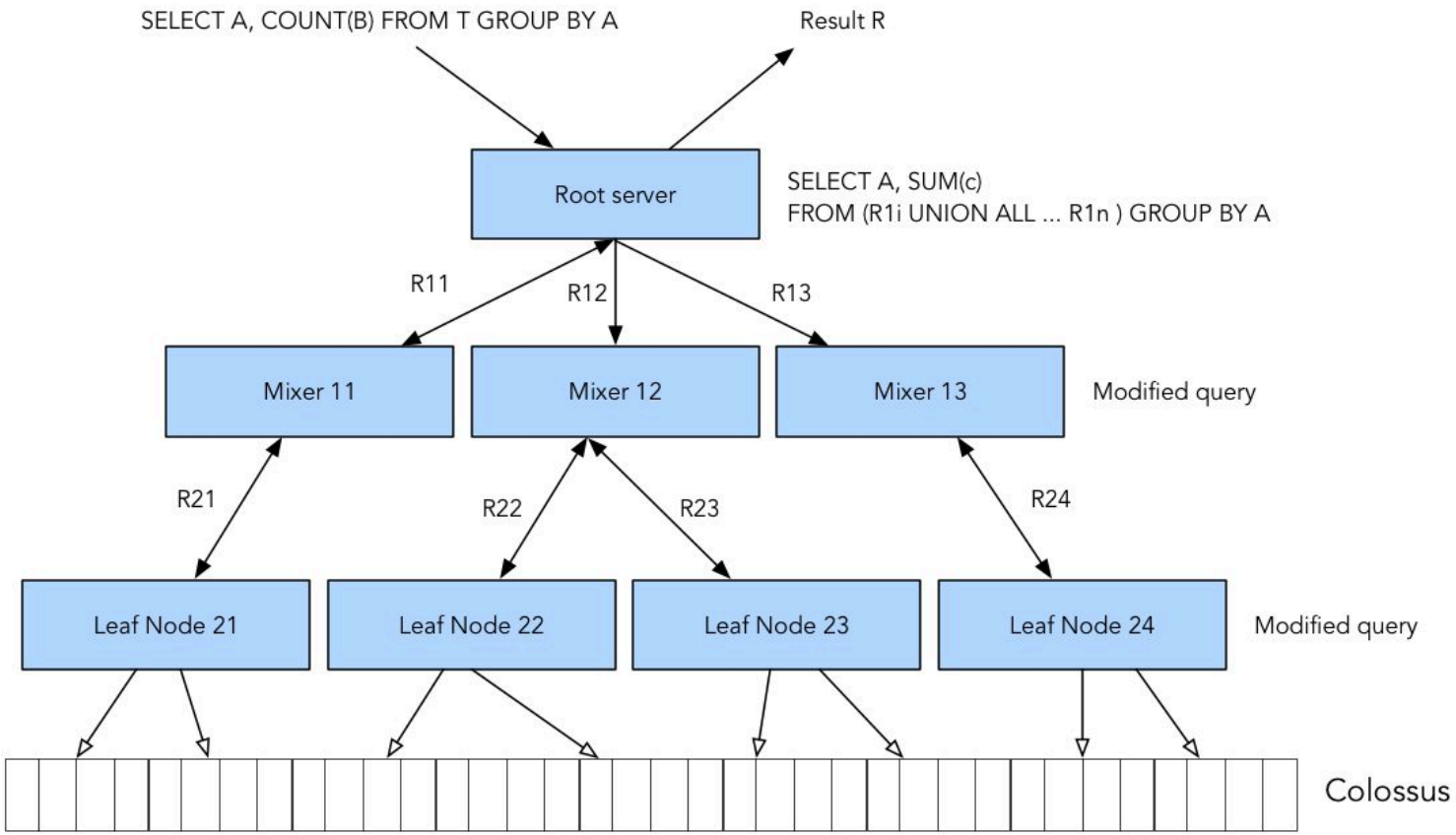
Managed Fault Tolerance

Historical illustration of replicated storage



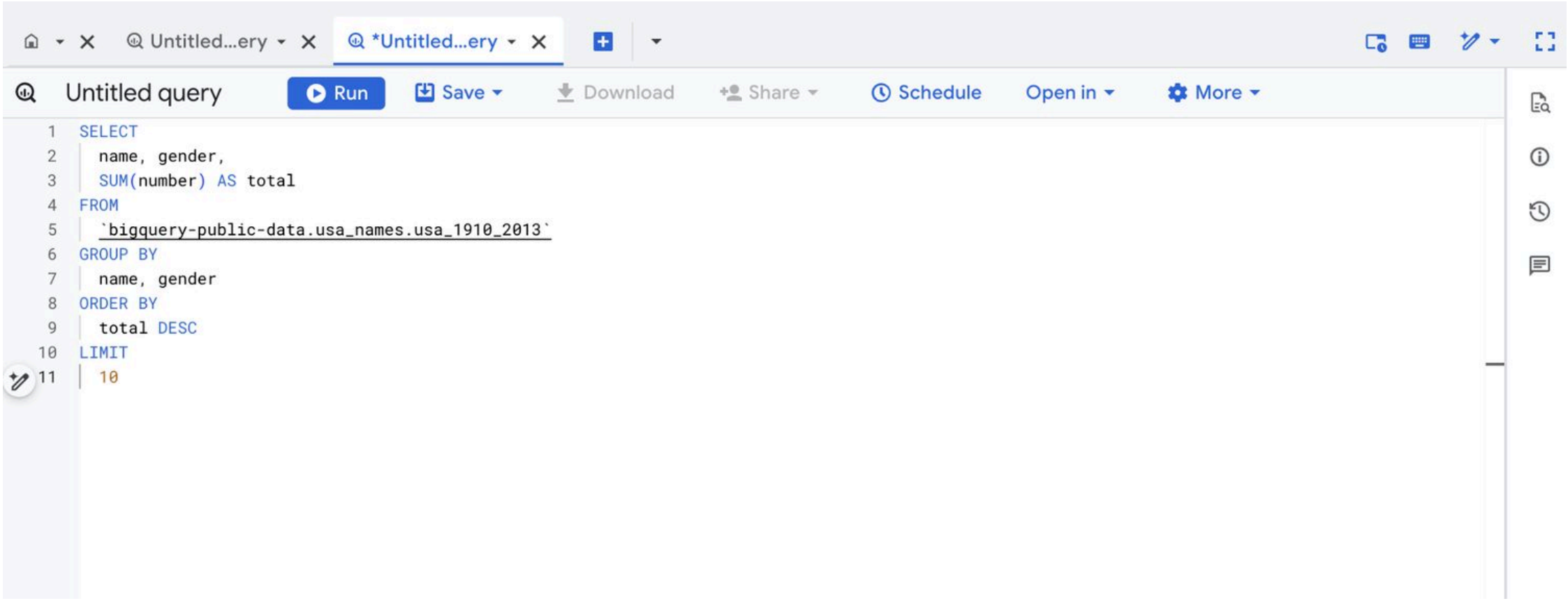
Replicated storage protects against infrastructure failures rather than incorrect analytical logic

Distributed Query Execution



Workers read portions of the table and combine partial results

SQL over Distributed Execution

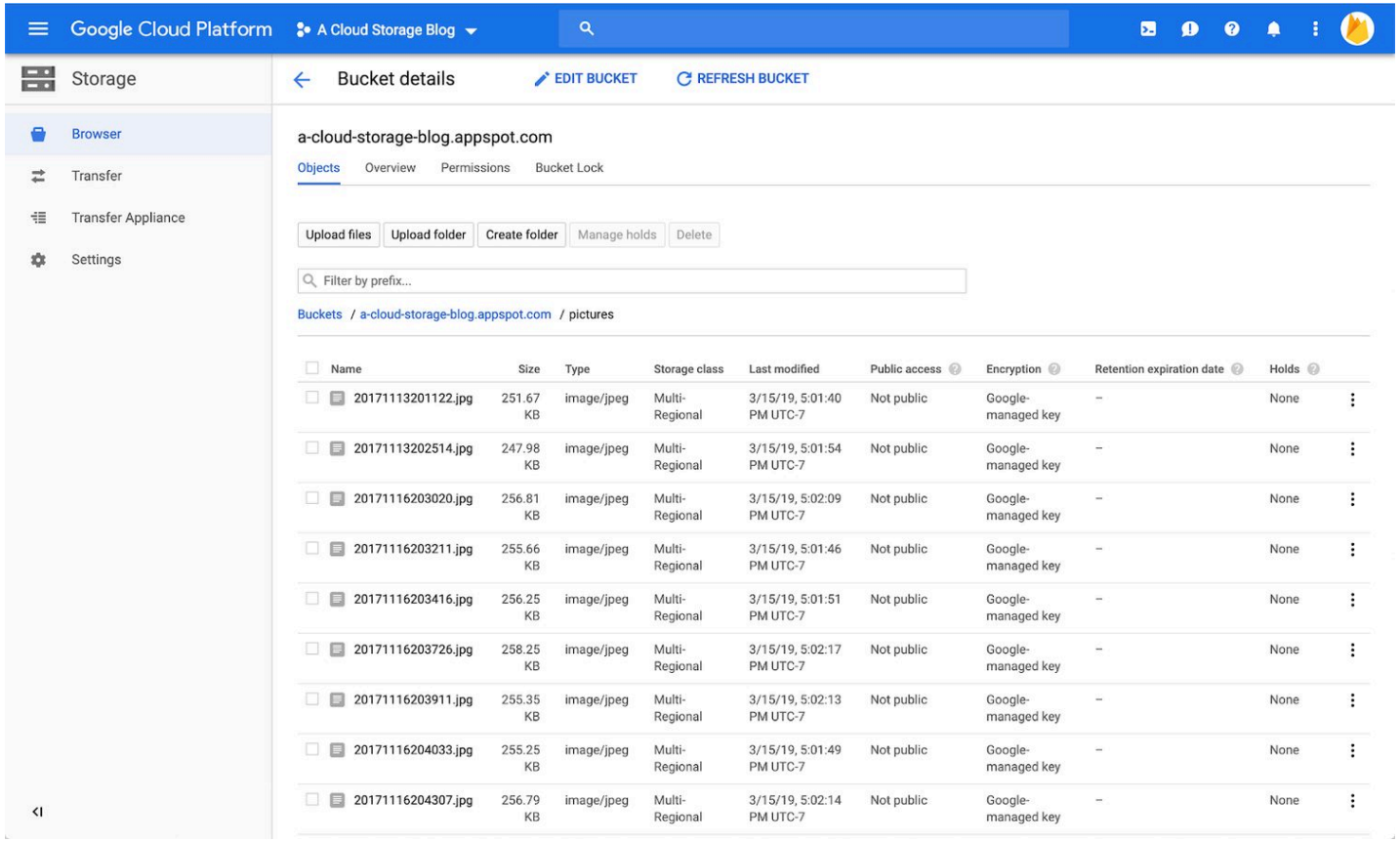


The user writes SQL; the service plans and runs distributed analytical work

03 · Lakes and Lakehouses

01	Cloud and Serverless	Who operates the infrastructure?
02	Analytical Warehouses	How are many rows read and summarized?
03	**Lakes and Lakehouses**	Where do files become tables?
04	Data Orchestration	Which task runs after which, and safely again?

Why Keep a Data Lake



Source files can remain in object storage before analytical tables are defined

Warehouse, Mart and Lake

Store	Organizing unit	Typical purpose
Warehouse	Integrated analytical tables	Consistent organizational analysis
Data mart	A subject or department subset	Focused reports
Lake	Files and datasets in varied formats	Preserve and reuse diverse inputs

A mart narrows the analytical scope; a lake broadens the retained source material

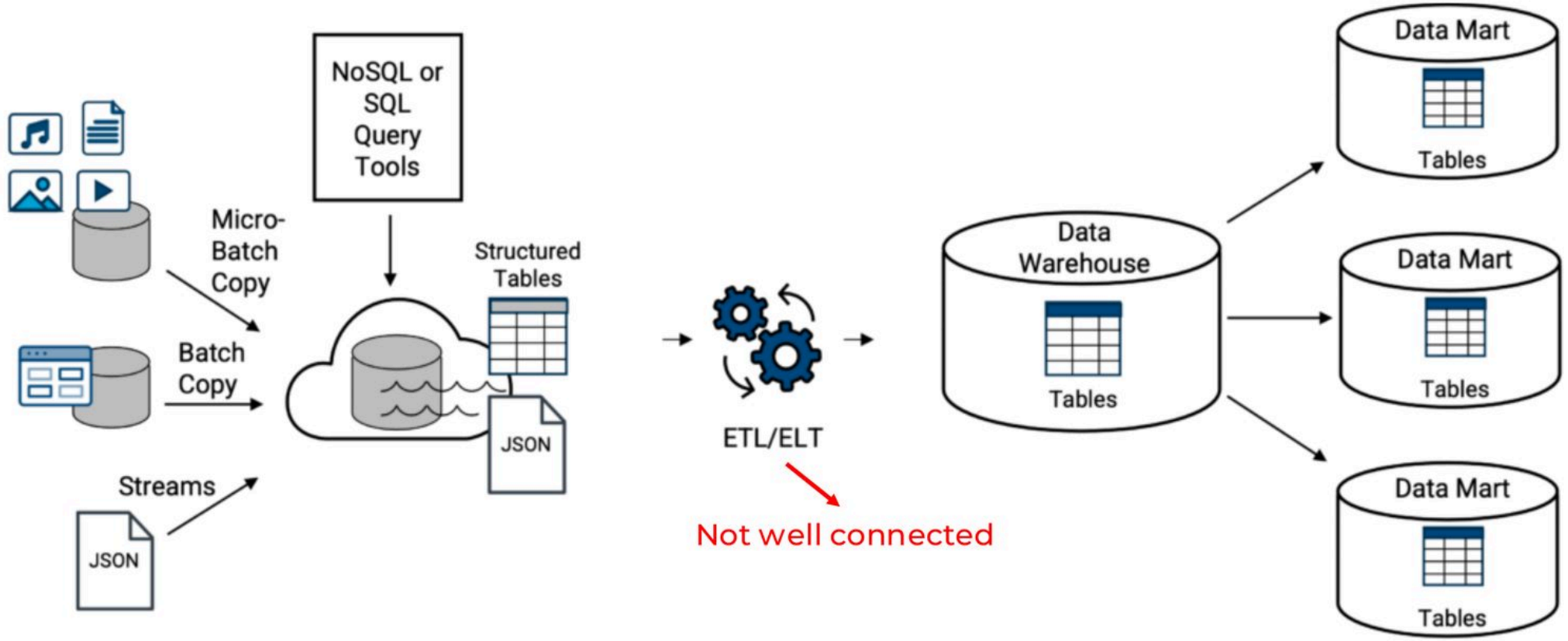
Schema on Read

Schema on read applies structure at use time; schema on write validates it when loading



Files can arrive first; structure and types are still required when the data is used

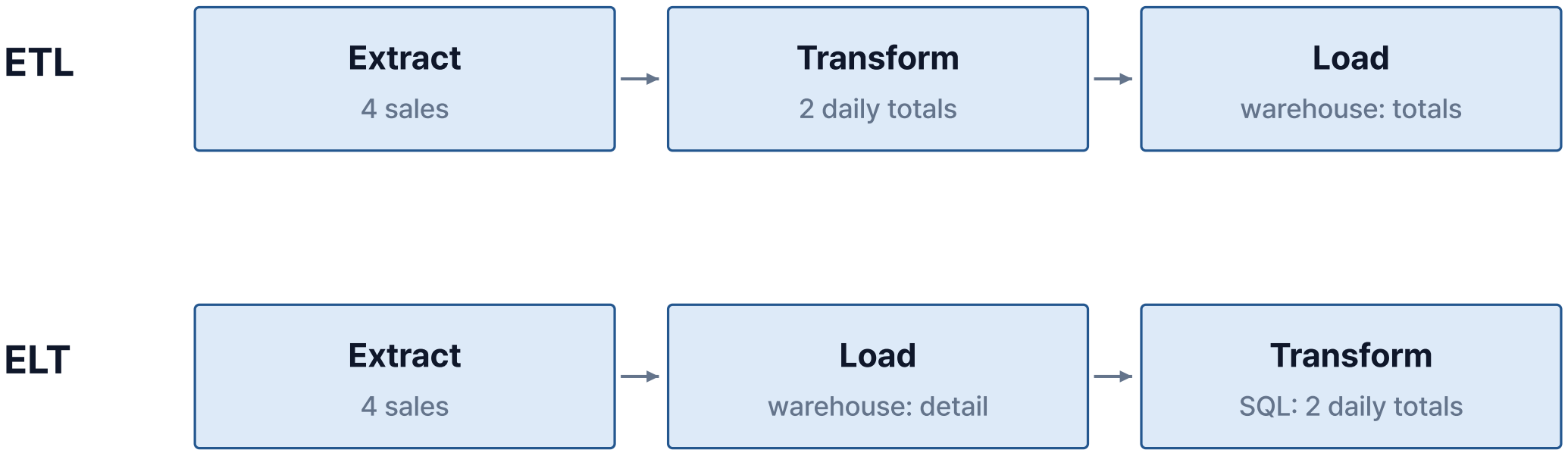
From Lake to Warehouse



Separate raw and analytical stores introduce copying, transformations and refresh dependencies

ETL and ELT

Extract, Transform, Load (ETL) or Extract, Load, Transform (ELT)



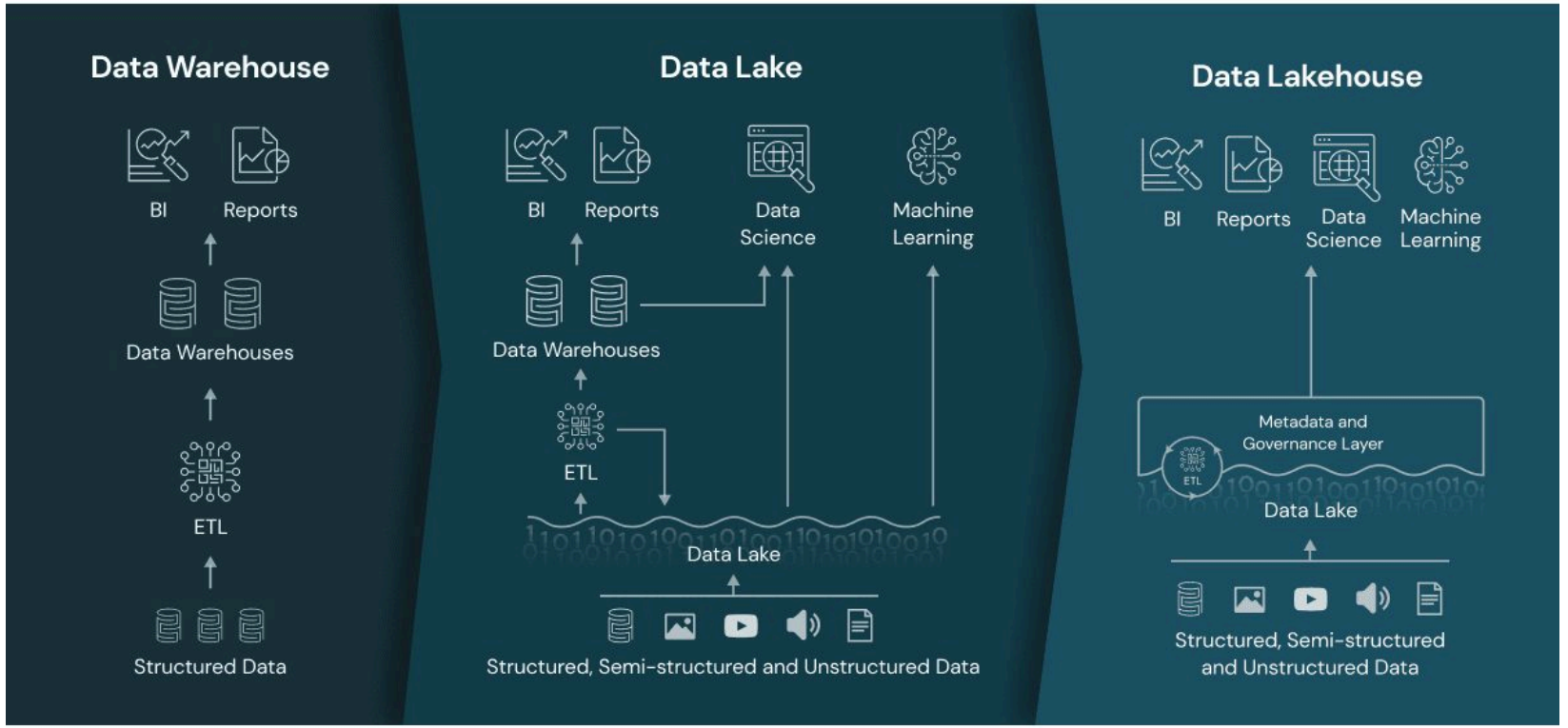
Transform before loading, or load first and transform inside the analytical store

Choosing ETL or ELT

Condition	Typical choice	Reason
Sensitive fields must stay outside the target	ETL	Remove them before crossing the boundary
Target compute is limited	ETL	Transform using upstream processing
Restricted raw area and scalable SQL compute	ELT	Load detail, then build analytical models inside

Choose the location by data boundaries and compute; decide detail retention separately

Why a Lakehouse



Lakehouse adds table management over lake files for shared analytical use

What Makes a Lakehouse Work

Layer	Mechanism	Reason
Table management	Transactions and versioned metadata	Readers see a consistent table
Query execution	Columnar files, pruning and statistics	SQL scans avoid unrelated data
Shared access	SQL and DS/ML use the same snapshot	Fewer independent data copies

Table transactions, query execution and shared data access turn files into managed tables

Tools Around a Lakehouse

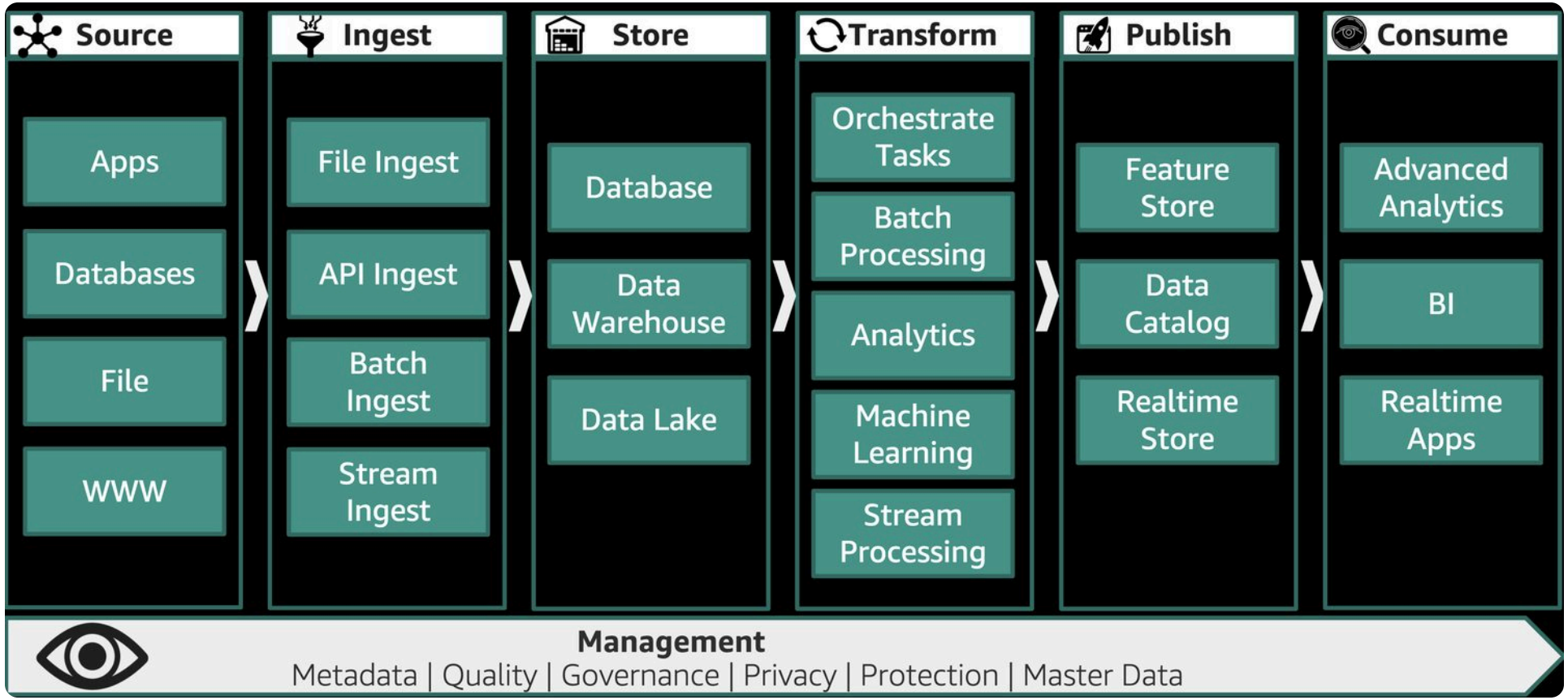
Compute and platform	**Spark, Databricks**	run transformations and queries
Table format	**Iceberg, Delta Lake**	manage table metadata and changes
Columnar file format	**Parquet**	encode values in files
Object storage	**S3, Cloud Storage**	store the data files

Storage, file format, table format and compute belong to different layers

04 · Data Orchestration

01	Cloud and Serverless	Who operates the infrastructure?
02	Analytical Warehouses	How are many rows read and summarized?
03	Lakes and Lakehouses	Where do files become tables?
04	**Data Orchestration**	Which task runs after which, and safely again?

Why Data Orchestration



A successful query can still run before its required data is ready

Airflow at Airbnb

2014
started at Airbnb by M. Beauchemin

2015
announced as open source

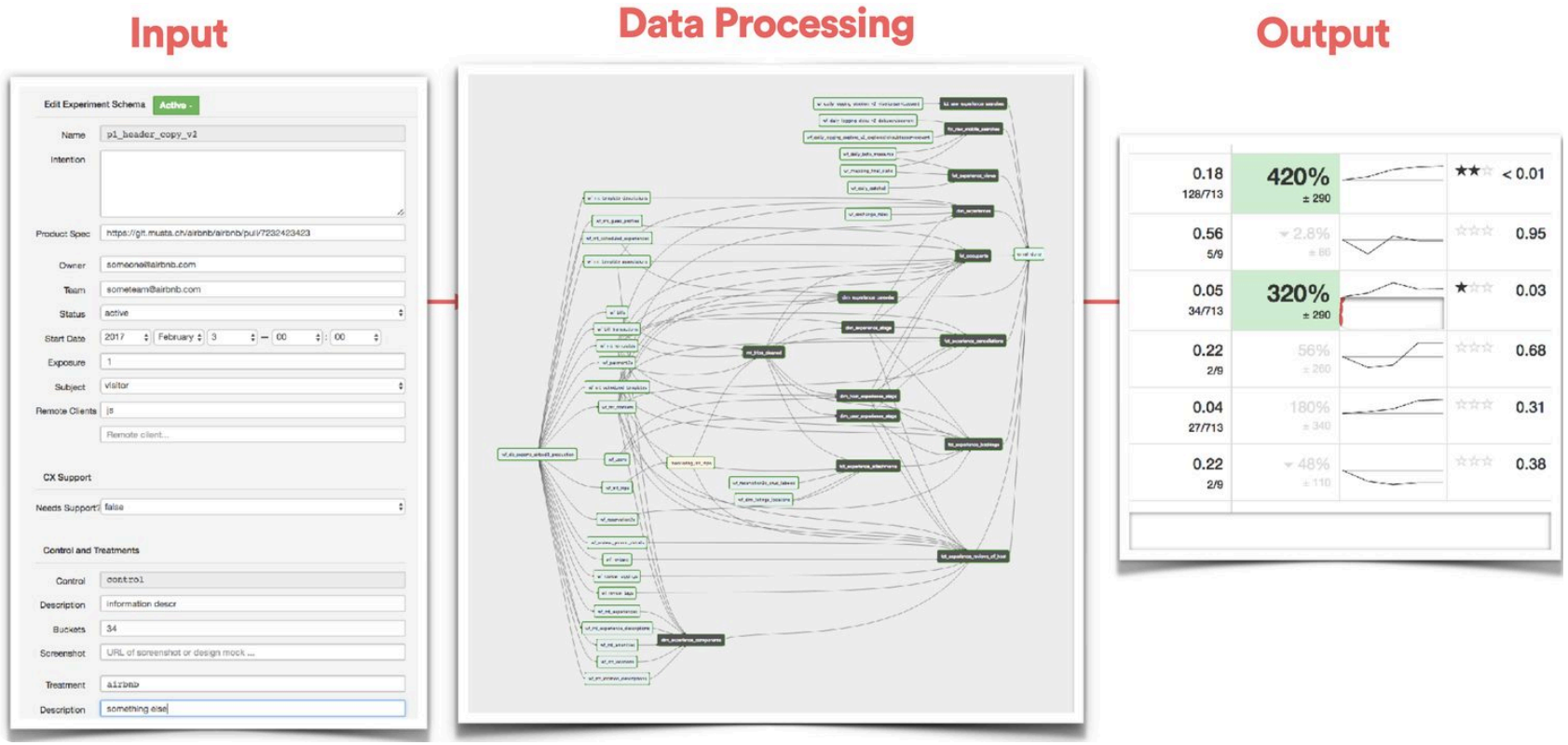
2016
Apache incubation

2019
Apache top-level project

Airflow grew from a company workflow tool into an Apache project

Case study: Airbnb Experiment Analytics

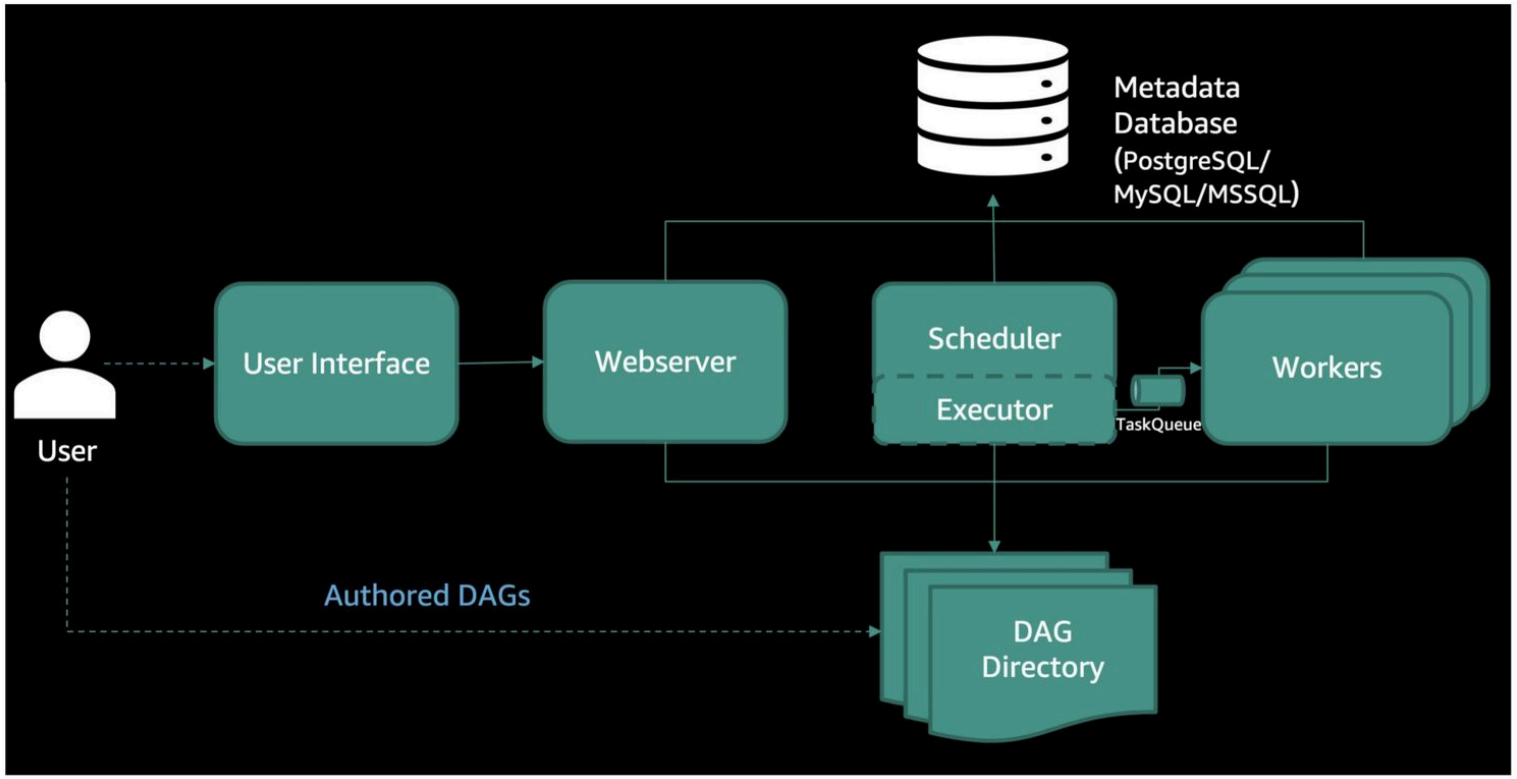
Historical Airbnb workflow: configuration, computation and experiment metrics



Experiment configuration drives a dependency graph that produces analytical results

Airflow Control and Workers

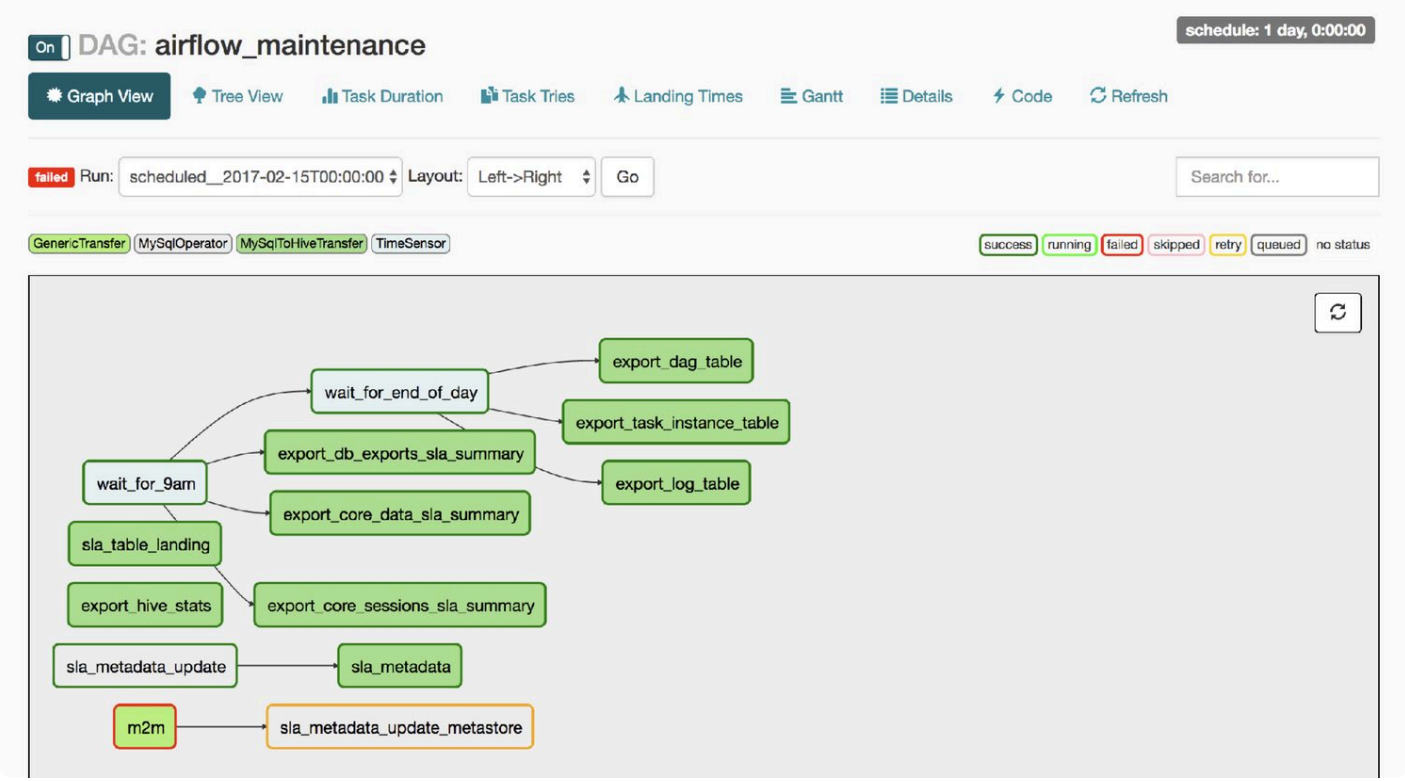
Historical architecture: focus on the responsibilities, rather than deployment details



Scheduling, task execution and recorded state are distinct responsibilities

Tasks and Dependencies

A Directed Acyclic Graph (DAG) defines tasks and their dependencies



Independent branches can run in parallel; dependent tasks wait for their upstream work

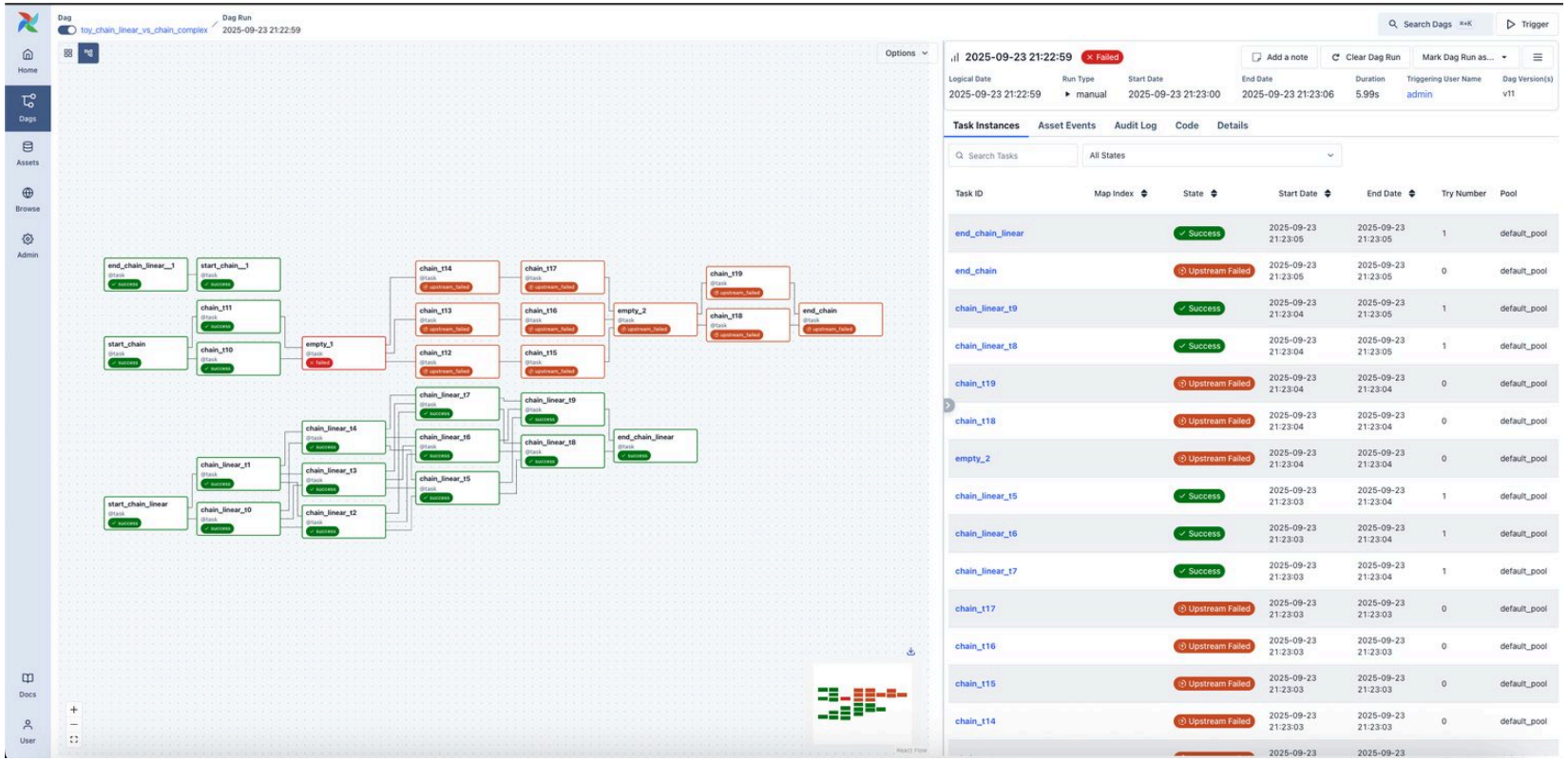
Input Readiness and Task Types

Concept	Role	Example
Task	A concrete unit of work in the DAG	Load the sales partition
Operator	A reusable definition for a task	Submit a warehouse SQL query
Sensor	A task that waits for a condition	Wait for the producer to publish a complete input

A sensor waits for an input contract; a clock time alone does not prove readiness

An Airflow Run

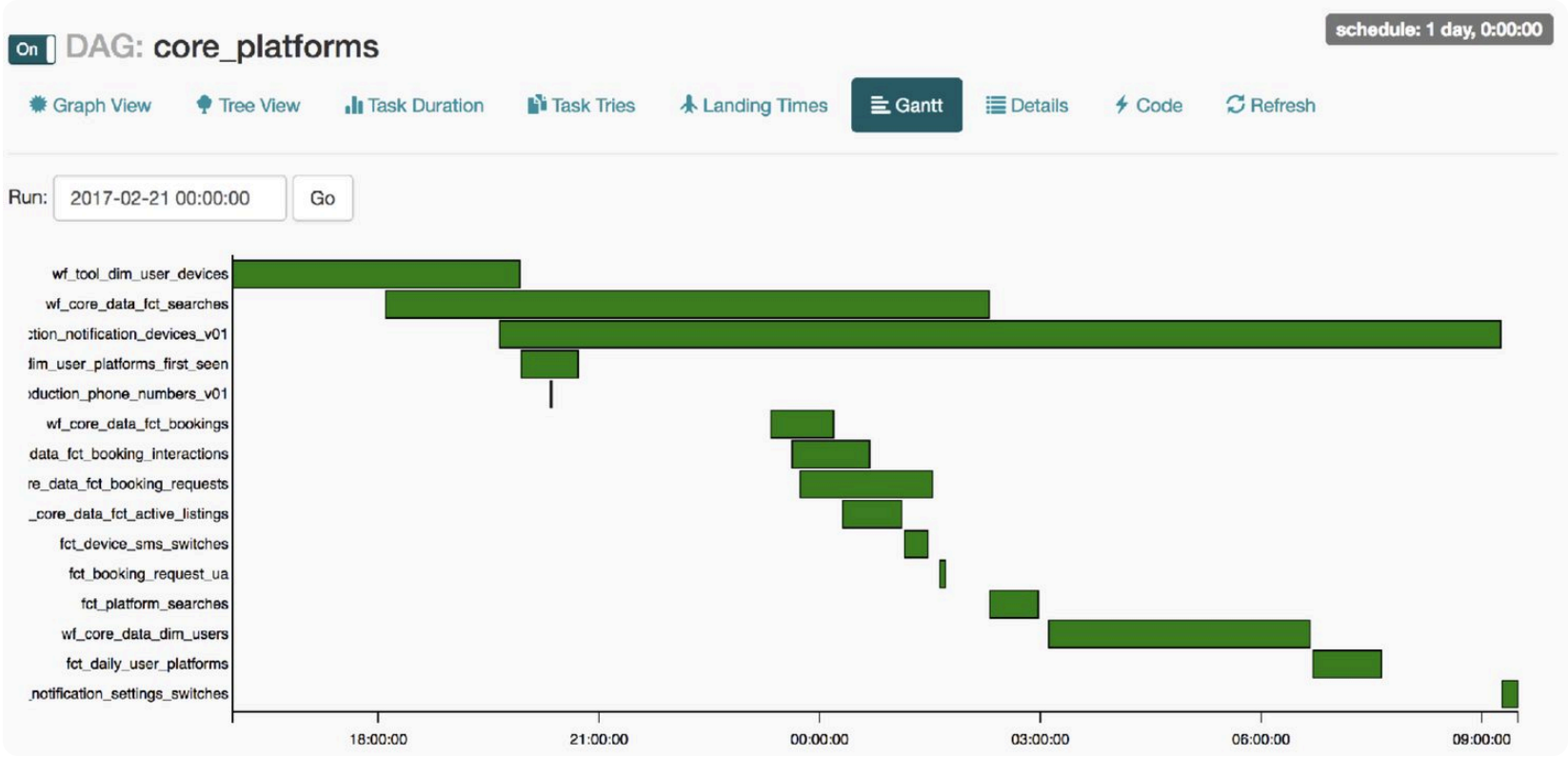
Apache Airflow interface



A real run shows succeeded, failed and waiting tasks

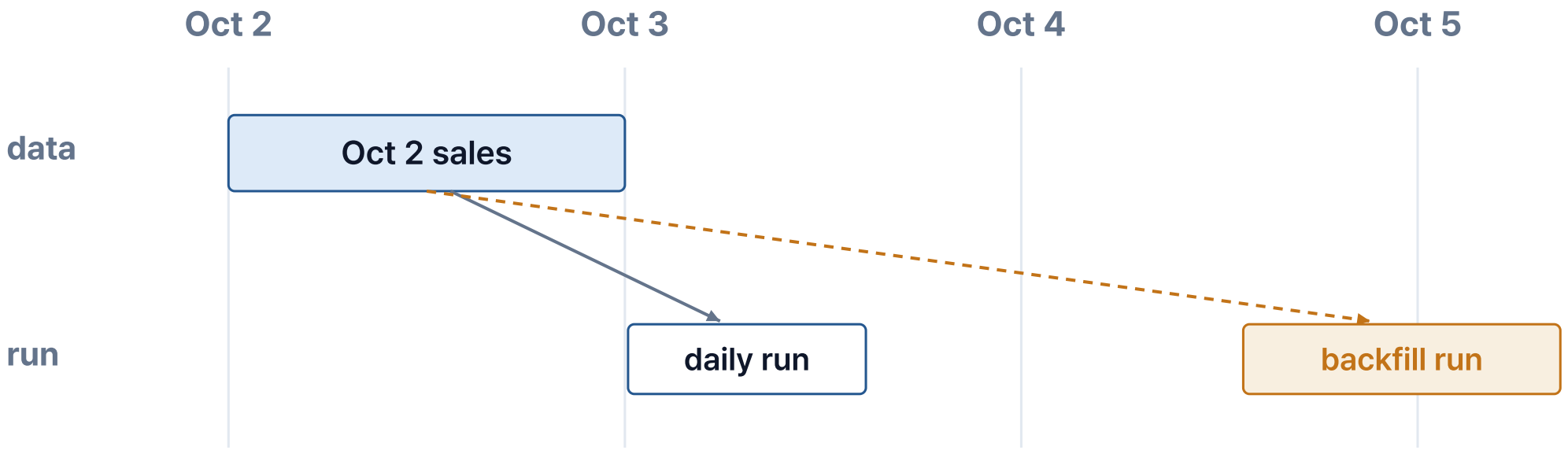
Read Execution Time in Gantt

Historical Airflow Gantt: actual overlap and long-running tasks



Use task durations together with dependencies to locate the path delaying completion

Data Interval and Run Time



The date being recomputed can differ from the time when the task runs

Retry and Backfill

Mechanism	Purpose	Required design
Retry	Run a failed or uncertain task again	Repeat without duplicating side effects
Backfill	Compute a selected historical interval	Use that interval, rather than the current time
Safe publication	Replace the prepared output after validation	Keep input version, target interval and commit boundary explicit

Orchestration schedules re-execution; the task must define safe, repeatable data changes

Takeaway: Tools Have Different Responsibilities

1. **Managed analytics** hides infrastructure while retaining storage and execution choices
2. **OLTP / OLAP** usually favor row / column stores; lakes retain inputs and lakehouses manage tables
3. **Airflow** coordinates task dependencies, input readiness and repeatable execution